

Android Dynamic Application Security Testing

Technology-Agnostic Practical Field Guide

From APK fingerprinting to runtime instrumentation, network interception, security-control analysis, exploit validation, and evidence.

How to Use This Guide

[START HERE](#)

This guide is not a list of tools. It is a decision-making methodology. Every stage below depends on the one before it — skipping straight to "run known bypass scripts" is why dynamic testing feels inconsistent across apps. The rest of this guide follows this pipeline in order.

```
APK
↓
FINGERPRINT    what is this, technically?
↓
PREPARE       environment matched to what was found
↓
OBSERVE       baseline behavior, unmodified
↓
INSTRUMENT    targeted hooks based on the baseline
↓
INTERCEPT   network visibility
↓
MANIPULATE    controlled input changes
↓
VALIDATE      does behavior confirm a real boundary crossing?
↓
PROVE IMPACT  Five-Part Vulnerability Validation Test
↓
REPORT
```

Every major technical section in this guide follows the same internal structure, so you always know where to look for a given answer:

[WHAT](#)[WHY](#)[IDENTIFY](#)[TEST](#)[OBSERVE](#)[CONFIRM](#)[ESCALATE](#)[IMPACT](#)

ELEMENT	ANSWERS
What / Why	What are we testing, and why does it matter?
Identify	How do we determine which specific implementation is in play?
Test	What do we actually do — the command or hook to run
Observe	What evidence should appear if this is working as expected?
Confirm	What proves the behavior actually changed, not just that nothing crashed?
Escalate	What's the next technique if the first approach fails?
Impact	When does this become a real, reportable vulnerability?

The APK is untrusted client code running in an environment the attacker fully controls. Every local security decision — is this device rooted, is this certificate valid, is this user authorized — is made on hardware the attacker owns. This guide exists to find out what happens when that control is actively exercised.

★ The First 60 Minutes

QUICK START

A fast-start sequence for the beginning of an assessment — a starting rhythm, not a replacement for the full methodology that follows. Every block points to the fuller section covering it.

00–10 MIN	10–20 MIN	20–30 MIN	30–40 MIN	40–50 MIN	50–60 MIN
Fingerprint Unzip and scan for framework signals, confirm ABI, check debuggable status, skim the manifest for exported components and schemes.	Environment Confirm root/emulator status. Launch fresh — no Frida yet. Record what happens unmodified. This is your baseline.	Attack Surface Map the primary flow (login) end to end: UI → local processing → IPC → network → storage → background.	Network Configure proxy, install CA (user store first), watch what traffic appears with no bypass attempted yet.	Runtime Instrumentation Attach Frida, enumerate classes, hook one method, confirm the observe/hook loop actually works.	Initial Security Tests Try one exported component via ADB, note WebView JS/bridge state, check SharedPreferences/logcat for plaintext secrets.

Do not start bypassing root, SSL pinning, or anti-Frida controls unless the control is actually blocking a security-relevant test. Bypassing something because it's there, not because it's in the way of a specific test you need to run, is how hours disappear without producing findings.

★ Tool Selection Matrix

[QUICK START](#)

Not a tutorial — a quick reference for which tool matches which task, and why.

TASK	PRIMARY TOOL	ALTERNATIVE	PURPOSE
APK inspection	jadx / apktool	androguard (scripted)	Decompile and read source/resources statically
Runtime instrumentation	Frida	Objection (Frida wrapper)	Hook, observe, and manipulate runtime behavior
Network interception	Burp Suite	mitmproxy	Intercept and tamper with HTTP(S) traffic
Native analysis	Ghidra	radare2	Disassemble and analyze native .so libraries
IPC	ADB (am, content, pm)	Drozer (optional, scripted)	Discover and invoke exported components
Logs	logcat	Android Studio Logcat view	Correlate runtime behavior with app-side output
WebView	Chrome remote debugging	Frida hooks on WebView methods	Inspect live JS execution and bridge behavior
Backup/data testing	adb backup (legacy, illustrative)	Device/OEM-specific backup review	Assess actual data exposure through the applicable mechanism

No single row here is a full methodology on its own — every tool listed is covered in context, with its actual limitations, in the numbered sections that follow. Android/ADB platform tooling is the primary approach throughout this guide; specialized tools like Drozer are always optional convenience layers, never required dependencies.

Table of Contents

Every major section follows the What/Why/Identify/Test/Observe/Confirm/Escalate/Impact structure introduced on page 2.

QUICK START

★ The First 60 Minutes	4
★ Tool Selection Matrix	5

01 · FOUNDATION

1 What Is Dynamic Testing?	8
2 Dynamic Testing Methodology	8
3 Baseline Before Instrumentation	8

02 · FINGERPRINTING & ATTACK SURFACE

4 Identify the Application Stack	10
5 Build a Runtime Profile	12
6 Runtime Attack Surface Mapping	12

03 · ENVIRONMENT & DEVICE

7 Android Test Environment	14
8 Root Detection	16
9 Emulator Detection	23
10 Anti-Debug / Anti-Instrumentation	24

04 · RUNTIME INSTRUMENTATION

11 Frida Fundamentals	26
12 Java/Kotlin Instrumentation	27
13 Native / JNI Analysis	28

05 · NETWORK INTERCEPTION

14 Network Testing Architecture	30
15 SSL/TLS Pinning Decision Tree	31
16 Framework-Specific Networking	33
17 Static → Dynamic Bridge	34

06 · RUNTIME ATTACK SURFACE

18 Authentication Testing	36
19 Biometric & Keystore Testing	37
20 Runtime Cryptography Testing	38
21 Authorization Testing	39
22 IPC Testing	40
23 Deep Link Testing	42
24 WebView Testing	43
25 Runtime Data Lifecycle	44
26 Backup & Restore Reality Check	45

07 · ADVANCED RUNTIME DEFENSES

27 Anti-Tampering & Integrity	47
28 Anti-Frida / Anti-Hooking	49
29 When Should I Stop?	49

08 · VALIDATION & REPORTING

30 Proving a Vulnerability	51
31 Dynamic Evidence Collection	52
32 False Positive Filter	52
33 Dynamic Finding Template	53
34 Final Testing Checklist	53

09 · WHEN THINGS DON'T WORK

35 Troubleshooting Decision Trees	55
-----------------------------------	----

APPENDIX & CLOSING

Practical Command & Fast-Path Reference	60
Field Card	62
Disclaimer	63

PART 01

Dynamic Testing Foundation

The mental model this entire guide is built on — what dynamic testing reveals that static analysis can't, and why baseline behavior matters before any instrumentation begins.

S1 What Is Dynamic Testing?

S2 Dynamic Testing Methodology

S3 Baseline Before Instrumentation

01 What Is Android Dynamic Application Testing?

	STATIC ANALYSIS	DYNAMIC ANALYSIS
What you examine	Decompiled code, manifest, resources	The running process, its memory, its network traffic
Reveals	What the app <i>could</i> do	What the app <i>actually</i> does under specific conditions
Model	Code/Config → Potential weakness → Validate	Input → Runtime behavior → Security control → Manipulation → Observed result → Impact
Misses	Runtime-only state (decrypted values, live tokens)	Anything never triggered during the test session

The APK is untrusted client code running in an environment the attacker fully controls. Any decision the app makes locally is made on hardware the attacker owns. Dynamic testing exists to find out what happens when that control is actively exercised.

02 Dynamic Testing Methodology

Every later stage in the pipeline (page 2) depends on the one before it. The most common cause of unreliable dynamic testing is skipping straight to "manipulate" without fingerprinting, preparing, or observing a baseline first.

03 Baseline Before Instrumentation

Do not start by hooking everything. Run the application normally first, and record startup behavior, permissions, network traffic, the authentication flow, storage writes, logcat, component launches, WebView behavior, crashes/warnings, and root/emulator behavior — before changing anything.

BASELINE → CHANGE ONE VARIABLE → OBSERVE THE DIFFERENCE → IDENTIFY THE SECURITY CONTROL

This single discipline is the difference between "I ran ten Frida scripts and three did something" and "I know exactly which control I bypassed and what changed as a result."

PART 02

Fingerprinting & Attack Surface

Identifying the framework does not by itself tell you how its security controls are implemented — this part builds the profile everything downstream depends on.

S4 Identify the Application Stack

S5 Build a Runtime Profile

S6 Runtime Attack Surface Mapping

04 Identify the Application Stack

TECHNOLOGY	STATIC INDICATOR	RUNTIME CONFIRMATION	INSTRUMENTATION LAYER	RISK OF ASSUMING WRONG
Java/Kotlin	Only classes.dex, no unusual native libs	Standard ART, traditional Activity lifecycle	Java.perform() — direct	Low — default case most tooling assumes
Flutter	lib/*/libflutter.so	Single-Activity shell, custom rendering	Native hooks into the .so — confirm HTTP client first	Assuming all networking is native-only; some use Cronet/ platform channels
React Native	assets/ index.android.bundle	JS bridge init in logcat at startup	Usually standard Java.perform() still applies	Assuming OkHttp — depends on RN version and architecture
Xamarin / .NET MAUI	lib/*/libmonodroid.so	Mono runtime init in logcat	Managed hooking limited; OS/proxy-layer often more reliable	MAUI may default to a different HTTP handler than classic Xamarin.Forms
Unity	libunity.so, libil2cpp.so	Unity splash, minimal native Views	Native hooks against IL2CPP-compiled code	Assuming Java hooks reach game logic — they don't
Cordova / Ionic	assets/www/, cordova.js	WebView is the entire UI	Java hooks + WebView/JS bridge focus	Treating it like native app networking
Cronet (any framework)	org.chromium.net, libcronet.so	HTTP/3-QUIC traffic possible	Different hook targets than OkHttp	Assuming OkHttp-specific hooks apply
Native C/C++	Heavy custom .so, many JNI declarations	Java classes are near-empty wrappers	Native-first: Interceptor.attach(), Ghidra	Assuming meaningful logic lives in Java

Fingerprint the framework

PURPOSE

Determine which runtime/networking model this app uses before selecting an instrumentation strategy.

COMMAND

```
unzip -l target.apk | grep -E "libflutter|libmonodroid|libunity|libil2cpp|index.android.bundle|cordova.js|libcronet"
```

EXPECTED OBSERVATION

One or more matching library/asset names, or none (→ likely plain Java/Kotlin).

NEXT

Match the result against the table above to select the instrumentation layer for later sections.

05 Build a Runtime Profile

Application:

Package:

Framework: Java/Kotlin | Flutter | React Native | Xamarin/.NET MAUI | Unity | Cordova | Native | Hybrid

Networking Stack: OkHttp | Cronet | Native | Framework-managed | Unknown

Root Detection: Present/Absent — category if known

Emulator Detection: Present/Absent — category if known

SSL Pinning: Present/Absent — implementation if known

Integrity Checks: Present/Absent

Every scenario in the troubleshooting section (§35) traces back to skipping this step. This profile is what you'd hand to a colleague picking up the assessment mid-way.

06 Runtime Attack Surface Mapping

USER ACTION → ACTIVITY/FRAGMENT/COMPOSE → LOCAL PROCESSING → IPC (if any)
→ WEBVIEW (if any) → NETWORK REQUEST → SERVER → RESPONSE
→ LOCAL STORAGE → BACKGROUND COMPONENT (refresh, sync, notification)

Action: Login

Network: POST /login

Storage: Access token → SharedPreferences; refresh token → ?

Background: Silent refresh observed ~50 min later

Logout: DELETE /session — confirm old token actually rejected

PART 03

Environment & Device Analysis

Root and emulator detection, and anti-debugging — the controls that decide whether you can test at all before you get to test anything else.

S7 Android Test Environment

S8 Root Detection

S9 Emulator Detection

S10 Anti-Debug / Anti-Instrumentation

07 Android Test Environment

FACTOR	EMULATOR	PHYSICAL DEVICE
Setup speed	Fast, snapshot/reset easily	Slower, harder to reset state
Root access	Straightforward on most images	Requires unlocking bootloader + flashing
Hardware-level detection	Frequently fails native/hardware checks	Passes by default — it's real hardware
Play Integrity API	Commonly lower-trust verdicts — depends on device state, config, and environment	More likely high-trust, but not automatic — config still matters

Start on emulator for speed. If detection resists Java-level bypass, confirm whether it's genuinely native/hardware-level before assuming a bypass is impossible. Physical ARM64 hardware is a time-efficiency escalation, not proof the native check can't be bypassed.

Before-Testing Sequence

A copyable ten-step checklist to run once, at the start of every assessment.

1-4 · Connect, Install, Launch

```
adb devices
adb shell getprop ro.product.cpu.abi | list adb install target.apk
adb shell monkey -p com.app.target -c android.intent.category.LAUNCHER 1
```

EXPECTED

Device listed as "device" state; ABI list matches the APK's native libs; clean install; app launches without a bypass in place — this IS your baseline.

5-9 · Proxy, CA, Verification, Logcat

```
adb shell settings put global http_proxy <burp_ip>:8080 # Install Burp CA via
Settings > Security > Install from storage
adb shell am start -a android.intent.action.VIEW -d "https://example.com"
adb logcat -c && adb logcat > baseline.log &
```

EXPECTED

Browser traffic appears in Burp first — if it doesn't, fix proxy/CA before testing the app. Then confirm app traffic separately.

NEXT

If browser worked but the app didn't, that's an app-specific finding (§14–15), not a setup problem.

10 · Confirm Root Status

```
adb shell su -c "id"
```

EXPECTED

uid=0(root) on a rooted test device — confirms your own environment before blaming the app.

08 Root Detection

WHAT

IDENTIFY

TEST

ESCALATE

IMPACT

What we're testing: whether the app's root check is client-side-only and bypassable, and — separately — whether bypassing it actually exposes something meaningful.

APPLICATION BEHAVIOR



IDENTIFY TRIGGER → CLASSIFY DETECTION → LOCATE IMPLEMENTATION



JAVA OR NATIVE?

JAVA

Instrument the specific Java API (File/Package/Property/Exec)

NATIVE

Locate exported symbol or offset, hook at the native layer (§13)



RETRY → VALIDATE (functionality reachable, not just "warning gone")

Step 1 — Establish the Baseline

Confirm the block and find where it happens

```
adb shell su -c "id" adb shell getprop ro.build.tags adb shell mount | grep " / "
adb shell pm list packages | grep -i "magisk\|supersu\|superuser"
```

EXPECTED

uid=0(root); "test-keys" on custom builds; "rw" on the mount; root-management packages visible to PackageManager.

NEXT

Launch the app and note exactly what happens — crash, warning dialog, or silent feature restriction. This determines your starting point.

Step 2 — Identify the Detection Category

CATEGORY	EVIDENCE	LIKELY IMPLEMENTATION	WHERE TO HOOK
File check	Reads /system/xbin/su or similar	File.exists()	Hook selectively for the specific path
Package check		PackageManager.getPackageInfo()	Hook the specific method

Queries installed
packages

System property	Reads build properties	SystemProperties.get()	Hook the property getter
Command execution	App shells out	Runtime.exec(), ProcessBuilder	Intercept the specific command
Native check	No Java-level match found	JNI-declared native method	Move to native workflow (§13)

Step 3 — Java-Level Bypass: File Check

PASS 1 — Observe Only

WHY THIS HOOK

Surface the exact path(s) checked — do not guess in advance

LIMITATION

Purely observational; makes no behavior change

```
Java.perform(function () {
    var File = Java.use("java.io.File");
    File.exists.implementation = function () {
        var path = this.getAbsolutePath();
        var result = this.exists();
        console.log("[*] File.exists() checked: " + path + " -> " + result);
        return result;
    };
});
```

EXPECTED OUTPUT

The exact path(s) checked — commonly /system/xbin/su, /system/bin/su, or /sbin/su specifically.

PASS 2 — Suppress Only the Exact Path

WHY THIS HOOK

Suppress only the exact path from Pass 1 — never match on a substring like "su" or "magisk," which also matches unrelated paths

```
Java.perform(function () {
    var File = Java.use("java.io.File");
    var TARGET_PATH = "/system/xbin/su"; // exact path observed in Pass 1
    File.exists.implementation = function () {
        var path = this.getAbsolutePath();
        if (path === TARGET_PATH) {
            console.log("[*] Suppressing File.exists() for: " + path);
            return false;
        }
        return this.exists();
    };
});
```

WHAT IT OBSERVES

Whether the warning disappears with this exact path suppressed

LIMITATION

A different warning appearing means a second, independent check exists — repeat Pass 1

Do not globally return false for every `File.exists()` call, and do not match on broad substrings. `File.exists()` is used constantly for unrelated purposes — a blanket or substring-based suppression breaks the app in unrelated ways and makes it impossible to tell whether you bypassed the right check.

Step 4 — Full Worked Example

Say the app blocks because it checks for `/system/xbin/su` specifically:

1. Confirm via Step 1 that this file exists on your rooted device (`adb shell ls -la /system/xbin/su`).
2. Load the Step 3 Pass 2 hook, with `TARGET_PATH` set to the exact string observed in Pass 1.
3. Relaunch spawned fresh: `frida -U -f com.app.target -l hook.js --no-pause`.
4. Retry the flow that previously triggered the block.
5. **Observe:** warning disappears, or a different block appears (→ second independent check, classify separately)?
6. Confirm the app proceeds into functionality that was genuinely gated.

Step 5 — Package Detection

```
Java.perform(function () {
  var PM = Java.use("android.app.ApplicationPackageManager");
  PM.getPackageInfo.overload(
    "java.lang.String", "int"
  ).implementation = function (pkg, f) {
    if (pkg.indexOf("magisk") !== -1) {
      throw Java.use(
        "android.content.pm.PackageManager$NameNotFoundException"
      ).$new();
    }
    return this.getPackageInfo(pkg, f);
  };
});
```

OBSERVES

Forces the exception a genuinely-absent package would produce.

Step 6 — System Property

```
Java.perform(function () {
    var SP = Java.use("android.os.SystemProperties");
    SP.get.overload(
        "java.lang.String"
    ).implementation = function (key) {
        var r = this.get(key);
        if (key === "ro.build.tags" &&
            r.indexOf("test-keys") !== -1) {
            return "release-keys";
        }
        return r;
    };
});
```

OBSERVES

Spoofs the specific property value the check reads.

Step 7 — Runtime.exec / Shell Command Detection

Observe which shell command the app checks for

```
Java.perform(function () {
    var Runtime = Java.use("java.lang.Runtime");
    Runtime.exec.overload("java.lang.String").implementation = function (cmd) {
        console.log("[*] Runtime.exec called with: " + cmd);
        if (cmd === "which su" || cmd === "su") {
            throw Java.use("java.io.IOException").$new("Command not found");
        }
        return this.exec(cmd);
    };
});
```

WHY THIS HOOK

The log line alone is often the fastest way to discover which command the app checks — run unsuppressed first, purely to observe.

Step 8 — Native Root Detection

Java check → native method (declared `native`) → JNI → .so → actual root check

1-4 · Locate the check

```
unzip -l target.apk | grep "\.so$" strings lib/arm64-v8a/libtarget.so | grep -i  
"su\|magisk\|root" readelf -s lib/arm64-v8a/libtarget.so | grep FUNC
```

NEXT

Identify architecture, recon strings, list exported symbols for a plausibly-named function.

5-6 · Hook the exported symbol

```
var lib = Process.findModuleByName("libtarget.so");  
var target = Module.findExportByName(lib.name, "isDeviceRooted");  
Interceptor.attach(target, {  
  onLeave: function (retval) {  
    console.log("[*] Native isDeviceRooted returned: " + retval);  
    retval.replace(0);  
  }  
});
```

LIMITATION

Only works for exported symbols. Stripped binaries need Ghidra → locate function → calculate offset → hook by address instead of name. Materially higher effort — flag in scoping.

Step 9 — If the Hook Doesn't Fire

`Java.use()` throws "class not found" or
hook never triggers



Class loaded? → `enumerateLoadedClasses()`



Overload matches? → `enumerateMethods()`



Declared native? → move to native workflow



Multiple ClassLoaders? → try each explicitly



Re-check Step 2's classification

Step 10 — Validation

"The warning disappeared" is not sufficient. Confirm all of:

- App proceeds past the block
- Gated functionality is actually reachable
- The security-sensitive operation can genuinely be tested
- Any downstream server-side control still gets evaluated

Vulnerability vs. Hardening Observation

Root detection bypass $\neq$ security vulnerability

Root detection bypass

- + local secrets now accessible that weren't before
- + no server-side check backing the same decision

= potentially meaningful finding

A root check paired with server-side attestation (Play Integrity API — current terminology; SafetyNet Attestation is deprecated) is materially different: bypassing the client side doesn't establish how the server treats the resulting integrity verdict.

09 Emulator Detection

CATEGORY	EXAMPLE SIGNAL
Build properties	Build.FINGERPRINT containing "generic", Build.MODEL containing "sdk"/"Emulator"
Hardware indicators	Missing IMEI, missing telephony radio
Sensors	Absent accelerometer/gyroscope
Filesystem artifacts	Genymotion-specific files, /dev/socket/qemud

Hook and spoof — implementation-dependent, not guaranteed

WHY THIS HOOK

Java-level Build.* checks are often straightforward to instrument, since they're typically plain field reads — but this depends on the implementation, not guaranteed.

```
Java.perform(function () {  
    var Build = Java.use("android.os.Build");  
    Build.FINGERPRINT.value = "google/walleye/walleye:9/PQ3A/real-device-string";  
    Build.MODEL.value = "Pixel 2";  
    Build.MANUFACTURER.value = "Google";  
});
```

EXPECTED OUTPUT

Retry the flow. Warning gone → this field alone was responsible. Persists → spoof one field at a time to isolate which mattered.

Java-level Build.* spoofed, still detected



Determine the remaining signal source

Native code — strings lib.so | grep "cpuinfo|qemu"

/proc inspection — adb shell cat /proc/cpuinfo

Hardware characteristics (missing IMEI/radio)

Sensor absence — grep SensorManager null-checks



If reproducing the environment becomes disproportionately expensive, use physical ARM64/device farm as a time-management escalation — not proof the native check can't be bypassed.

10 Anti-Debugging & Anti-Instrumentation

Debug.isDebuggerConnected()

```
Java.perform(function () {
    var D =
    Java.use("android.os.Debug");
    D.isDebuggerConnected
        .implementation = function () {
            return false;
        };
});
```

TracerPid: native checks often read `/proc/self/status` for a non-zero TracerPid.

```
adb shell cat /proc/<pid>/status | grep
TracerPid
```

Frida detection — process-name, port (27042), library/module, or `/proc` scanning:

```
adb shell mv /data/local/tmp/frida-
server /data/local/tmp/fs-renamed adb
shell /data/local/tmp/fs-renamed -l
0.0.0.0:31337 &
```

Determining signature vs. behavior-based: retest after rename+port change. Disappears → signature-based. Persists → behavior-based (timing, injection artifacts), needs Frida Gadget or a different approach.

Do not assume renaming defeats all detection. It resolves common signature checks only. Confirm what the check actually protects before spending effort bypassing it — a low-value UI feature is a different priority than a payment flow.

PART 04

Runtime Instrumentation

Frida as an instrumentation platform, not a bag of bypass scripts — the fundamentals every later section builds on.

S11 Frida Fundamentals

S12 Java/Kotlin Instrumentation

S13 Native / JNI Analysis

11 Frida Fundamentals for Security Testing

CONCEPT	WHEN TO USE	LIMITATION
attach vs spawn (-f)	Spawn when a check runs very early	Spawn can be affected by run-as restrictions on non-debuggable apps
Method hooks + overloads	Any method with multiple overloads	Wrong overload signature = silent no-op, not an error
Class enumeration	Confirm a class is loaded before debugging further	Doesn't show which ClassLoader if multiple exist
Native module enumeration	Locating native functions when Java hooks are insufficient	Only works for exported symbols

The First 10 Minutes With Frida

1-3 · Version check, list, spawn

```
frida --version adb shell /data/local/tmp/frida-server --version frida-ps -U
frida -U -f com.app.target --no-pause
```

NEXT

Version mismatch is the #1 cause of attach failures — confirm this first, always.

4-8 · Enumerate, hook, observe

```
Java.perform(function () {
  var Target = Java.use("com.app.auth.LoginManager");
  Target.login.implementation = function (user, pass) {
    console.log("[*] Arguments: " + user + " / " + pass);
    var result = this.login(user, pass);
    console.log("[*] Return value: " + result);
    return result;
  };
});
```

9-10

Remove the hook and repeat the same action to compare behavior with and without instrumentation — confirms the hook itself isn't altering the flow unexpectedly.

12 Java/Kotlin Runtime Instrumentation

TARGET	WHAT IT REVEALS
Authentication methods	Whether validation is client-side only or deferred to the server
Token handling	Predictability, storage location, transmission method
Crypto functions	Whether a static "weak crypto" finding is exploitable (§20 — observing ≠ finding)
Certificate validation	Which bypass approach applies (§15)

"Class Exists in JADX But Frida Can't Find It"

```
Java.use("com.app.Target") throws "class not found"
```

↓

Name exactly right? Inner classes use \$ not a dot.

↓

Dynamically loaded? Trigger the flow first, then retry.

↓

Different ClassLoader? enumerateClassLoaders(), try each.

↓

Obfuscated (ProGuard/R8)? Mapping may not match jadx output.

↓

Code path simply never executed (§35, Scenario 10)

13 Native / JNI Runtime Analysis

Java layer → JNI boundary → Native library (.so) → Security-sensitive operation

Signal you need this layer: a Java method resolves fine, has the right signature, but hooking it produces no observable effect — because it's declared native.

1-4 · Locate and recon

```
unzip -l target.apk | grep "\.so$" adb shell getprop ro.product.cpu.abi | list
strings lib/arm64-v8a/libtarget.so | grep -iE "root|debug|crypto|verify"
readelf -s lib/arm64-v8a/libtarget.so | grep FUNC
```

5-10 · Hook, observe, validate

```
var lib = Process.findModuleByName("libtarget.so");
var target = Module.findExportByName(lib.name,
    "Java_com_app_security_RootCheck_isRooted");
Interceptor.attach(target, {
    onEnter: function (args) {
        // args[0]=JNIEnv*, args[1]=jobject/jclass — applies to
        // standard JNI-exported methods found via this naming pattern,
        // NOT necessarily to arbitrary native functions found other ways
    },
    onLeave: function (retval) { retval.replace(0); }
});
```

LIMITATION

Stripped binaries have no export to hook by name. Escalate to Ghidra → locate function via strings from step 3 → calculate offset → hook by address.

PART 05

Network Interception

Six distinct failure modes get collapsed into "SSL pinning" colloquially.
Distinguishing them is the actual troubleshooting skill.

§14 Network Testing Architecture

§15 SSL/TLS Pinning Decision Tree

§16 Framework-Specific Networking

§17 Static → Dynamic Bridge

14 Android Network Testing Architecture

Android App → Networking Stack → TLS Layer → Proxy (Burp) → API

SYMPTOM	DIAGNOSIS
Browser traffic fails too	Proxy configuration problem — not app-specific
Browser works, app fails	App-specific — likely pinning or a stack ignoring the system proxy
Only HTTPS fails, HTTP works	TLS/CA trust/pinning issue specifically
Only one domain fails	Domain-specific rule — check <code>network_security_config.xml</code>
Some requests fail, others succeed	Multiple networking stacks in one app (§35, Scenario 9)
No traffic at all, even errors	Possible QUIC/HTTP3, custom protocol, or fully native stack

15 SSL/TLS Pinning Decision Tree

SYSTEM CA TRUST $\neq$ CERTIFICATE PINNING

Six distinct failure modes get collapsed into "SSL pinning" colloquially:

#	FAILURE MODE
1	No network traffic at all
2	Traffic exists but doesn't pass through the proxy
3	TLS handshake failure — fails before any cert is evaluated
4	Certificate trust failure — CA isn't trusted
5	Certificate pinning failure — CA trusted, specific cert/key doesn't match
6	Application-layer rejection after successful TLS

Burp configured, CA installed



Traffic visible?

YES

Continue to API testing (§18+)

NO

CA trust → networking stack → validation layer



Identify the validation layer

Java / OkHttp CertificatePinner

WebView (separate stack)

Flutter / React Native

Native TLS callback



Instrument the matching layer → retry

RESOLVED

Continue to application testing

STILL BLOCKED

Consider category 6 (app-layer rejection);
Conscript hook as one more data point — not
universal; escalate to native (§13)

Implementation-Specific Workflows

OkHttp CertificatePinner

IDENTIFY

```
grep "CertificatePinner" jadx_src/
```

```
// Confirm the actual overload first —  
// signatures vary by OkHttp version  
Java.perform(function () {  
    var CP =  
    Java.use("okhttp3.CertificatePinner");  
    var ov = CP.check.overloads;  
    for (var i=0;it.className).join(",  
    "));  
});
```

LIMITATION

Applying the wrong overload is a silent no-op, not an error — always enumerate first.

Custom X509TrustManager

IDENTIFY

```
grep -A3 "implements X509TrustManager" jadx_src/
```

```
var Custom = Java.use(  
    "com.app.network.CustomTrustManager"  
);  
Custom.checkServerTrusted.overload(  
  
    "[Ljava.security.cert.X509Certificate;",  
    "java.lang.String"  
).implementation = function (chain,  
    at) {  
    console.log("[*] Check suppressed");  
};
```

LIMITATION

Hook the specific implementing class by name — a generic interface hook won't reach it.

WebView, Network Security Config, Flutter

WEBVIEW

grep "onReceivedSslError" jadx_src/ → hook to force handler.value.proceed(). Specific to WebView traffic only — the main app stack needs its own separate bypass.

NETWORK SECURITY CONFIG

cat res/xml/network_security_config.xml — confirm a <pin-set> actually applies to the domain under test before assuming pinning applies to it.

FLUTTER

Do not reach for one universal offset. Identify the actual networking package (dio, http, cronet_http), confirm architecture/version, instrument the appropriate native layer. State explicitly that offsets are version/architecture-dependent.

16 Framework-Specific Network Testing

FRAMEWORK	IDENTIFY	VALIDATION LAYER	COMMON FAILURE
Java/Kotlin + OkHttp	grep "okhttp3"	CertificatePinner, custom TrustManager	A second pinning layer exists (e.g. WebView pins separately)
Flutter	libflutter.so present	Commonly BoringSSL in the .so — confirm the HTTP client first	Function offsets vary by Flutter version/architecture
React Native	index.android.bundle	Often native OkHttp under the JS bridge — not guaranteed	A custom native module pins independently
Xamarin / .NET MAUI	libmonodroid.so	Managed ServerCertificateValidationCallback or native handler	AOT compilation obscures method names
WebView	WebViewClient override	The engine's own TLS stack, separate from main app	Treating WebView traffic as part of the main stack
Native (raw sockets)	Custom .so, minimal Java networking	Native certificate verify callback	Stripped symbols require offset-based hooking

17 Static → Dynamic Bridge

Every static finding is a *lead* — the dynamic step confirms whether it's reachable and exploitable.

STATIC (VOL. 01)

addJavaScriptInterface()



DYNAMIC

Can attacker-controlled content actually reach the bridge?

STATIC (VOL. 01)

CertificatePinner present



DYNAMIC

Does it pin the traffic path you care about, or only some domains?

STATIC (VOL. 01)

getIntent().getData()



DYNAMIC

Can a separate app invoke this with attacker-controlled URI data?

STATIC (VOL. 01)

Sensitive value in
SharedPreferences



DYNAMIC

Is it actually present there during a real session — for how long?

STATIC (VOL. 01)

new Random() near token gen



DYNAMIC

Does it actually influence a security-relevant token, or just UI jitter?

STATIC (VOL. 01)

android:allowBackup="true"



DYNAMIC

Can sensitive data actually be recovered through the real mechanism? (§26)

A static finding without dynamic confirmation is a lead. A dynamic observation without static context is an isolated data point. Both guides meet in the middle at a confirmed finding (§30).

PART 06

Runtime Application Attack Surface

Authentication, authorization, IPC, deep links, WebView, and data — where the actual business-logic findings live.

[§18](#) Authentication · [§19](#) Biometric/Keystore

[§20](#) Crypto · [§21](#) Authorization

[§22](#) IPC · [§23](#) Deep Links · [§24](#) WebView

[§25](#) Runtime Data · [§26](#) Backup

18 Authentication Testing

NORMAL REQUEST → CAPTURE (Burp) → MODIFY → REPLAY → OBSERVE → VALIDATE

FLOW	MANIPULATION
Logout	Capture pre-logout request. Logout in-app. Replay original in Repeater — 200 instead of 401 confirms no server-side revocation.
Token expiration	Capture near expiry, wait past it, replay. Acceptance past expiry is the finding.
Password reset	Request reset, attempt token reuse, attempt from a different session context.
OTP	Replay with guessed/adjacent values rapidly — missing 429/lockout is the finding.
Account switching	Check whether switching triggers real re-auth, or just a client-side ID swap on the same token.
MFA	Attempt to replay the post-MFA authenticated state without completing the second factor.

Always distinguish **client-side bypass** (skipped a UI step) from **server-side enforcement** (the server independently checked). A finding requires the server accepting something it shouldn't.

19 Biometric & Keystore Runtime Testing

Biometric prompt → Authentication callback → CryptoObject/Keystore key → Sensitive operation

The distinction that matters: does the success callback alone unlock the operation, or does the app release a Keystore key requiring authentication binding?

ILLUSTRATIVE INSTRUMENTATION EXAMPLE

```
Java.perform(function () {  
    var Callback = Java.use("androidx.biometric.BiometricPrompt$AuthenticationCallback");  
    Callback.onAuthenticationSucceeded.implementation = function (result) {  
        console.log("[*] Forcing biometric success callback");  
    };  
});
```

Forcing this callback changes app state — it does **not** by itself prove a bypass. The only real finding is the protected operation succeeding without valid biometric-backed authorization ever occurring.

Practical flow: identify the call site → determine if a CryptoObject exists → inspect the callback → test manipulation → attempt the sensitive operation → determine whether a Keystore key was actually required → validate cryptographic enforcement.

20 Runtime Cryptography Testing

Input → Transformation → Key retrieval → Crypto operation → Output

Common call sites (exact classes are implementation-dependent)

```
Java.perform(function () {
  var Cipher = Java.use("javax.crypto.Cipher");
  Cipher.doFinal.overload("[B").implementation = function (input) {
    console.log("[*] Cipher.doFinal input: " + input);
    var result = this.doFinal(input);
    console.log("[*] Cipher.doFinal output: " + result);
    return result;
  };
});
```

OBSERVATION	LIKELY SOURCE
Same bytes every time, across restarts	Hardcoded in code or resources
Only accessible via AndroidKeyStore provider	Keystore-backed — not extractable this way
Different every session, tied to login	Server-provided, session-specific

Observing a key in memory is not automatically a vulnerability. Determine where it came from, why it's accessible, what capability an attacker needs to reach the same point, and what can be done with it.

21 Authorization Testing

- **Horizontal** — can Account A access Account B's data at the same privilege level?
- **Vertical** — can a lower-privilege account reach higher-privilege functionality?
- **IDOR/BOLA** — does changing an ID parameter return another user's data?

Client-side authorization is never a trusted security boundary. Every finding must be confirmed at the API layer directly via Burp Repeater.

SEQUENCE	STEPS
Horizontal (IDOR)	Capture User A's request → substitute User B's ID in Repeater → replay → compare response
Vertical	Low-privilege account attempts a request identified (via traffic diffing or static review) as admin-only
Hidden UI button	Underlying API call still present — invoke directly, bypassing the UI gate
Client-side role flag	Hook or edit the local flag, call the API directly — does the server independently re-check?

22 IPC Testing

PROPERTY	WHAT IT MEANS
Exported	The component can be invoked by another process at all
Permission-protected	Invoking process must hold a specific Android permission
Authenticated	The app requires session/login state regardless of invocation route
Authorization-protected	Beyond authentication, requires a specific role/privilege

`exported="true"` alone is not a finding — the finding requires the actual reachable gap. **ADB/platform tooling is the primary approach throughout**; Drozer is an optional convenience layer for scripted enumeration, never a required dependency.

DISCOVER → INSPECT PERMISSIONS → INVOKE (separate process where possible)
→ MANIPULATE inputs → OBSERVE → VALIDATE authorization

Inspect permissions before invoking

```
adb shell dumpsys package com.app.target | grep -B2 -A5 "AdminPanelActivity"
adb shell dumpsys package com.app.target | grep -A3
"com.app.target.permission"
```

EXPECTED

"permission=" line's absence alongside `exported=true` is the combination worth investigating. `protectionLevel=normal` is granted automatically; `dangerous` requires user grant; `signature` requires matching signing key.

Activities

```
adb shell am start -a
android.intent.action.VIEW \ -d
"content://com.app.target/data" \ -n
com.app.target/.DetailActivity \ --es
"user_id" "1042" --ez "is_admin"
"true"
```

OBSERVE

Launches (exported+reachable)? Shows sensitive screen directly, or redirects to login (internal auth check)?

Broadcast Receivers

```
adb shell am broadcast -n
com.app.target/.SessionReceiver \ --
es "session_token" "attacker-
controlled-value"
```

NOTE

Implicit broadcasts leak to any listening app on the device.

Services

```
adb shell am startservice -n
com.app.target/.SyncService --es
"account_id" "1042" adb shell dumpsys
activity services com.app.target
```

NOTE

Often higher-value — background/sync logic with less scrutiny than UI screens.

Content Providers

```
adb shell content query --uri
content://com.app.target.provider/
users --where "id=1 OR 1=1" adb shell
content update --uri content://.../
users --bind "role:s:admin" --where
"id=1042"
```

NOTE

Permissions often scoped per-path via <path-permission> — enumerate paths beyond the one found statically.

Genuinely separate process: ADB shell commands run from a distinct process/UID already — a reasonable proxy. For higher-confidence findings, package a minimal separate test APK calling the same Intent from a genuinely distinct installed app.

23 Deep Link Testing

Custom schemes vs App Links: a custom scheme (`myapp://`) has no ownership verification. An App Link requires `assetlinks.json` — a stronger guarantee, but verification can fail silently.

```
adb shell pm get-app-links com.app.target
```

Manipulation examples

```
adb shell am start -a android.intent.action.VIEW -d "myapp://reset?token=abc123&user=9999"
adb shell am start -a android.intent.action.VIEW -d "myapp://evil-host/reset?token=abc123"
adb shell am start -a android.intent.action.VIEW -d "myapp://reset?token=abc123#admin"
```

TEST

Host, path, query, fragment, stripped params, URL-encoding — each twice: once as the intended account, once as a different one.

Realistic targets: password reset, invitation links, payment confirmation, account switching, document/file access, any link loading a URL into a WebView.

24 WebView Testing

"JavaScript enabled" is not a finding. Neither is `addJavascriptInterface()` presence alone.

Where does the loaded URL come from?



Attacker-controllable? → Loaded? → JS enabled? → Bridge exposed?



Can the attacker-controllable content reach the bridge specifically, with real capability?

Practical inspection

```
chrome://inspect # connect device, select the app's WebView
```

IN DEVTOOLS CONSOLE

`Object.getOwnPropertyNames(Android)` lists what's reachable from JS

```
<script>Android.<identifiedBridgeMethod>(<controlled-test-input>);</script>
```

NOTE

Method name must come from actual enumeration, not assumed in advance.

25 Runtime Data — Sensitive Data Lifecycle

SOURCE → PROCESSING → MEMORY → NETWORK → STORAGE → LOGGING → BACKUP →
CLIPBOARD/SCREEN

Extraction per location

```
adb shell run-as com.app.target cat shared_prefs/user_prefs.xml adb shell run-  
as com.app.target cat databases/app.db > local.db && sqlite3 local.db  
".tables" adb logcat | grep -iE "token|password|auth|secret" adb shell  
screencap -p /sdcard/proof.png # black image confirms FLAG_SECURE
```

FALLBACK

run-as requires a debuggable app; otherwise root is required, or observe values purely via Frida hooks at the point they're written.

Value:	Session token
Storage:	SharedPreferences, plaintext
Backup:	allowBackup=true — see §26 for whether exploitable
Clipboard/Screen:	Not copyable, FLAG_SECURE present

26 Backup & Restore Reality Check

FLAG (allowBackup) → which mechanism applies? → dataExtractionRules restrictions?

- can sensitive data be extracted? → unlocked device + on-device tap required
- is extracted data itself still Keystore-protected? → IMPACT

Validation sequence

```
aapt dump xmltree target.apk AndroidManifest.xml | grep -i "allowBackup\|dataExtractionRules" adb backup -f loot.ab -noapk com.app.target java -jar abe.jar unpack loot.ab loot.tar find . -type f | xargs grep -l -iE "token|password|secret" 2>/dev/null
```

ILLUSTRATIVE — LEGACY MECHANISM, VERSION-DEPENDENT

adb backup targets the older Backup Manager mechanism and does **not** represent a universal modern Android backup method. Requires an unlocked device and manual on-device confirmation — not a silent remote extraction. The actual applicable mechanism must be identified separately for the target Android version.

PART 07

Advanced Runtime Defenses

"Modified APK accepted" is not automatically a vulnerability — the tester must determine whether the bypass removes a meaningful security boundary.

§27 Anti-Tampering & Integrity

§28 Anti-Frida / Anti-Hooking

§29 When Should I Stop?

27 Anti-Tampering & Integrity Checks

DETECTION → TRIGGER → EVIDENCE → LOCATION → INSTRUMENT → VALIDATE

Full Worked Example — Signature/Integrity Verification

API VERSION NOTE

Older applications commonly use `GET_SIGNATURES / PackageInfo.signatures`. Modern applications more commonly use `GET_SIGNING_CERTIFICATES / PackageInfo.signingInfo` instead — a different field. Inspect the app's actual API usage (step 4) before choosing which to hook.

1-4 · Repackage, install, observe, identify

```
apktool d target.apk -o modified/ && apktool b modified/ -o modified.apk
apksigner sign --ks test-keystore.jks modified.apk adb install modified.apk
grep -rn "GET_SIGNATURES\|GET_SIGNING_CERTIFICATES" jadx_src/
```

OBSERVE

Exact failure — crash on start, warning dialog, or silent single-feature failure — determines where the check sits.

6-8 · Instrument, observe expected-vs-actual, manipulate

```
Java.perform(function () {
    var PM = Java.use("android.app.ApplicationPackageManager");
    var GET_SIGNATURES = 64;
    PM.getPackageInfo.overload("java.lang.String", "int").implementation =
function (pkg, flags) {
    var info = this.getPackageInfo(pkg, flags);
    if ((flags & GET_SIGNATURES) !== 0) {
        console.log("[*] Actual signature: " +
info.signatures.value[0].toCharsString());
        // Once the expected value is known (step 7), substitute it:
        // var Signature = Java.use("android.content.pm.Signature");
        // info.signatures.value = [Signature.$new(expectedHex)];
    }
    return info;
};
});
```

LIMITATION

Manipulate only the specific call site identified — not every `PackageManager` method defensively.

9–11. Retry the modified APK with the hook active → confirm the protected functionality actually becomes reachable, not just that the warning disappeared → check whether server-side attestation (Play Integrity or similar) still independently blocks the modified app.

"Modified APK accepted" $\neq$ automatically a vulnerability

If server-side attestation independently rejects the modified APK, the client-side check being bypassable is a hardening note, not a standalone finding. If nothing backs it up and real functionality/data is exposed, write it up with the template in §33.

28 Anti-Frida / Anti-Hooking

Process/library/port detection is covered practically in §10 — the rename/port-randomize technique applies directly here too.

CATEGORY	HOW TO INVESTIGATE
Memory-region scanning	adb shell cat /proc/<pid>/maps grep -i frida — hook the file-read or the resulting decision, not the scan itself
Function-integrity checks	Compare behavior with a hook active vs. removed on the same run — a difference beyond your hook's intended effect signals integrity detection reacting to instrumentation

Do not maintain an ever-growing library of countermeasures. The useful skill is recognizing the pattern and deciding whether continued effort is justified (§29).

29 When Should I Stop Chasing a Bypass?

"Bypass a control only when it blocks a security-relevant test."

- ☐ The defense is explicitly outside the engagement's scope
- ☐ Native obfuscation makes further analysis disproportionately expensive relative to likely finding value
- ☐ The control, even bypassed, has no meaningful security impact
- ☐ The same underlying boundary can be tested directly through the API/server instead
- ☐ You already have sufficient evidence to assess the control without a full bypass
- ☐ The app uses commercial RASP/anti-tamper tooling requiring specialized vendor-specific approaches

Escalating effort indefinitely on a single control is not thoroughness — it's a scoping failure. Document what you tried, what you concluded, and move on.

PART 08

Validation & Reporting

Observation is not a vulnerability. This part is the difference between a hook that fired and a finding a developer can act on.

§30 Proving a Vulnerability

§31 Dynamic Evidence Collection

§32 False Positive Filter

§33 Dynamic Finding Template

§34 Final Testing Checklist

30 Proving a Vulnerability

OBSERVATION $\neq$ VULNERABILITY

Five-Part Vulnerability Validation Test

1. Attacker action
- + 2. Missing security control
- + 3. Security boundary actually crossed
- + 4. Concrete, articulable impact
- + 5. Reproducible without instrumentation itself being part of the attack path

= Validated Finding

The fifth condition matters: if a result only exists because Frida forcibly changed a value, ask whether that manipulation simulates something a real attacker could otherwise achieve — or is an artifact of the testing method with no real-world equivalent. **"Frida can change X" is not, by itself, a vulnerability statement.**

Not yet a finding: "The app stores the session token in SharedPreferences."

A finding: "The token is extractable via adb backup on an unlocked device, remains valid 30 days post-logout — confirmed via direct API replay, independent of instrumentation."

31 Dynamic Evidence Collection

EVIDENCE TYPE	WHY IT'S USEFUL TO DEVELOPERS
Screenshots	Visual proof for UI-observable issues
Burp request/response pairs	Exact reproduction data
Logcat output	Correlates timing with app-side behavior
Frida script output	Shows the exact hook and observed values
Affected endpoint / class / method	Points directly at the fix location
Clear reproduction steps	The single most useful artifact for a fast fix

32 False Positive Filter

INDICATOR	WHY INSUFFICIENT ALONE	WHAT MUST BE PROVEN
exported=true	Many exported components are intentional entry points	Sensitive functionality reachable without required auth/permission
JavaScript enabled	Required for essentially all modern web content	An exposed bridge reachable from attacker content with real capability
allowBackup=true	Depends on the mechanism and device state	Full chain walked per §26
MD5 usage	Not automatically exploitable — context-dependent	Used where collision/preimage properties actually matter
HTTP URL present	May be a non-sensitive third-party resource	Sensitive data actually transits that specific URL
Hardcoded string	Could be a dummy/test or non-sensitive identifier	It's a live credential with real, current access
debuggable=true	Only matters on the actual shipped release build	Confirmed this is the real production build
Root detection present	Detection existing isn't itself a finding	Bypassable and removes a control relied on elsewhere
Certificate pinning absent	Absence alone isn't a finding either	Realistic MITM position with actual data exposure confirmed

33 Dynamic Finding Template

```
TITLE
OBSERVED BEHAVIOR
ATTACKER ACTION
SECURITY CONTROL BYPASSED / MISSING
RUNTIME EVIDENCE          (Burp req/resp, Frida output, logcat, screenshots)
SECURITY BOUNDARY CROSSED
IMPACT
REPRODUCTION STEPS
REMEDiation
STATIC CORRELATION        (which Vol.01 finding does this confirm? – §17)
DYNAMIC VALIDATION        (states each part of §30's test explicitly)
```

34 Final Dynamic Testing Checklist

- ☐ Baseline recorded before instrumentation
- ☐ Framework identified, instrumentation confirmed applicable
- ☐ Runtime profile completed
- ☐ Attack surface mapped for primary flows
- ☐ Environment matches detection categories present
- ☐ Root/emulator/anti-debug fingerprinted before bypass
- ☐ Network interception through the correct validation layer
- ☐ Auth/biometric tested with server-side re-validation in mind
- ☐ Authorization tested directly at the API layer
- ☐ IPC tested from a genuinely separate process where possible
- ☐ Deep links tested with scheme vs App Link distinction
- ☐ WebView bridge exposure confirmed reachable
- ☐ Sensitive data lifecycle traced per critical value
- ☐ Backup exposure walked through the full mechanism chain
- ☐ Every finding passes the Five-Part Vulnerability Validation Test
- ☐ Evidence collected for every confirmed finding
- ☐ Every indicator checked against the false-positive filter
- ☐ Findings written using the template

PART 09

When Things Don't Work

The single most practical section in this guide. Ten real scenarios — symptom, check, result, next action, escalation, success criteria.

§35 Troubleshooting Decision Trees

1 Burp Cannot Intercept Traffic

SYMPTOM	No requests appear despite proxy configured.
CHECK	Does a plain browser work through the same proxy?
IF PASS	Only the app fails → app-specific (pinning, custom networking, HTTP/3-QUIC) → §15's six-category tree.
IF FAIL	Browser also fails → proxy/CA setup problem, not app-specific.
ESCALATE	tcpdump at the network level to confirm traffic leaves the device at all.
SUCCESS	Requests appear in Burp, correlated with the specific in-app action.

2 SSL Pinning Bypass Does Not Work

SYMPTOM	Bypass script runs without error; traffic still blocked.
CHECK	Which of the six §15 categories actually applies — re-run the full tree, don't assume pinning specifically.
IF NATIVE	TLS itself completed in Burp's logs? → app-layer rejection (category 6), not pinning.
ESCALATE	Apply the matching §16 layer; Conscript hook as an additional data point, not guaranteed; native investigation (§13) last.
SUCCESS	Decrypted request/response bodies visible for the previously blocked domain.

3 Frida Cannot Attach

SYMPTOM	frida-ps fails, or attach hangs/errors.
CHECK	Architecture and version match exactly? adb shell ps grep frida-server running? SELinux denials in logcat?
ESCALATE	Match versions; try spawn (-f); try a renamed binary if detection suspected. Frida Gadget if server-based attach is fundamentally blocked.
SUCCESS	frida-ps -U lists the process, or spawn+attach completes without immediate crash.

4 Application Detects Root

SYMPTOM	Warning, restricted functionality, or crash specifically on a rooted device.
CHECK	Toggle Magisk Hide/DenyList — does behavior change?
ESCALATE	Apply the fingerprint → bypass from §8. If server-side attestation (Play Integrity), document what was/wasn't testable client-side rather than claiming a full bypass.
SUCCESS	App proceeds into functionality that genuinely behaves as intended, not just "didn't crash."

5 Application Detects Emulator

SYMPTOM	Blocked functionality or crash specifically on emulator, not physical rooted device.
CHECK	Identical behavior on confirmed physical device? Spoof Build.* fields via Frida.
ESCALATE	If spoofing doesn't resolve it, confirm whether the check reads native-layer signals before concluding unbypassable — native hooking (§13) can sometimes still reach it.
SUCCESS	App behaves identically to confirmed real hardware.

6 Application Detects Frida

SYMPTOM	Crash, freeze, or behavior change only when Frida is attached.
CHECK	Does renaming frida-server + randomizing port change the outcome?
ESCALATE	Resolves → signature-based. Persists → behavior-based, needs Frida Gadget with reduced footprint. For heavily hardened apps, weigh continued effort against relying on static findings (§29).
SUCCESS	App runs its normal flow with hooks active and firing.

7 Java Hook Does Not Trigger

SYMPTOM	Script loads cleanly, class resolves, but the hook never fires.
CHECK	Overload signature matches exactly? <code>Java.enumerateMethods()</code> to confirm real signature; hook the constructor to confirm the class loads.
ESCALATE	Check for native declarations in the same class; enumerate <code>ClassLoaders</code> if multiple exist; move to §13 once native confirmed.
SUCCESS	Hook fires during the exact expected user action.

8 App Crashes After Instrumentation

SYMPTOM	Crash shortly after attach, or immediately after a specific hook fires.
CHECK	Logcat crash stack trace. Crashes on attach alone (before any hook)? Removing the hook prevents the crash?
ESCALATE	Fix type mismatches; spawn with early attach if timing-related. Attach-alone crash → treat as Scenario 6. Otherwise Gadget injection or static analysis + targeted native patching.
SUCCESS	App runs stably through the full intended test flow with hooks active.

9 Only Some Traffic Is Visible

SYMPTOM	Some requests appear in Burp, others don't.
CHECK	Which specific domains missing? Check network_security_config.xml for per-domain <pin-set> entries.
ESCALATE	Targeted hook for the responsible library, or the Conscript hook as a broader attempt. If an SDK pins independently by design (payment SDKs), document as intentional rather than forcing past it.
SUCCESS	All relevant traffic visible, or the remaining gap documented with a clear reason.

10 Function Exists Staticly But Never Executes

SYMPTOM	Method visible in decompiled source, hook never fires during normal use.
CHECK	Remote feature-flag/config SDK nearby? Try plausible preconditions — account tier, region via VPN, tampered server response in Burp.
ESCALATE	If confirmed genuinely unreachable, downgrade to a static-only note rather than a dynamically-proven finding.
SUCCESS	The function executes and becomes testable, or its unreachableity is explicitly documented.

ADB / Platform

Confirm ABI support

```
adb shell getprop ro.product.cpu.abi
```

PURPOSE / EXPECTED

No root required. Returns arch list e.g. arm64-v8a — tells you whether an ARM-only native library can load on this device.

Route traffic to Burp

```
adb shell settings put global http_proxy <burp_ip>:8080
```

PURPOSE / LIMITATION

No root required. Precondition, not a diagnostic — many apps ignore the system-wide proxy setting, especially custom networking stacks.

Establish system-level CA trust — ENVIRONMENT-DEPENDENT, NO SINGLE PROCEDURE

WHY IT VARIES

Dynamic partitions (Android 10+), Android Verified Boot, and SELinux can each block a direct /system remount depending on the device. Systemless rooting (Magisk) avoids modifying /system entirely, using its own module mechanism instead.

REQUIREMENTS

Root, plus identifying the device's actual rooting method (traditional vs. systemless) and Android version first — there is no universal command sequence.

LIMITATION

System CA trust ≠ certificate pinning bypass. This solves trust-failure (§15 category 4) specifically — it does nothing against genuine pinning (category 5).

Backup extraction — ILLUSTRATIVE, LEGACY MECHANISM

```
adb backup -f loot.ab -noapk com.app.target
```

REQUIREMENTS / LIMITATION

Unlocked device + manual on-device confirmation — not silent/remote. Not universal — varies by Android version and whether the app uses dataExtractionRules (API 31+). Identify the actual applicable mechanism separately (§26).

Check exported components – ADB (primary)

```
adb shell am start -n com.app.target/.AdminPanelActivity adb shell content query --uri content://com.app.target.provider/users
```

NOTE

ADB requires no specialized tooling. See Drozer below for an optional scripted alternative.

Confirm App Link verification state

```
adb shell pm get-app-links com.app.target
```

PURPOSE

autoVerify="true" doesn't guarantee verification occurred — this confirms actual status per domain.

Optional Specialized Tools

Drozer – OPTIONAL, SCRIPTED IPC ENUMERATION

```
dz> run app.activity.info -a com.app.target -u # optional, if Drozer available
```

NOTE

Never required — see the ADB-based exported-components card above for the primary approach. Drozer syntax varies across forks — confirm which fork before copying commands verbatim.

Objection – OPTIONAL, FAST-PATH TRIAGE

```
objection -g com.app.target explore --startup-command "android sslpinning disable"
```

LIMITATION

Silent console success doesn't mean traffic is visible — always verify against Burp directly.

Before Testing

- ☐ Fingerprint framework
- ☐ Confirm ABI
- ☐ Build runtime profile
- ☐ Establish clean baseline

Runtime

- ☐ Root/emulator behavior
- ☐ Frida attach/spawn
- ☐ Java/native layer
- ☐ IPC
- ☐ Deep links
- ☐ WebView
- ☐ Runtime data

Network

- ☐ Proxy configured
- ☐ CA trust confirmed
- ☐ Identify network stack
- ☐ Determine validation layer
- ☐ Verify traffic in Burp

Validation — Five-Part Test

- ☐ Attacker action
- ☐ Missing security control
- ☐ Security boundary crossed
- ☐ Concrete impact
- ☐ Reproducible without instrumentation itself being the attack path

When Blocked

```
Symptom
↓
Identify layer
↓
Test hypothesis
↓
Instrument minimum required layer
↓
Retry
↓
Escalate only if necessary
```

STATIC → DYNAMIC → CONFIRMED FINDING

DISCLAIMER

This material is intended for authorized security testing, research, education, and defensive security assessment.

Test only applications, devices, APIs, and infrastructure for which you have explicit authorization.